



Πανεπιστήμιο Πειραιώς
Σχολή Τεχνολογιών Πληροφορικής και Τηλεπικοινωνιών
Τμήμα Ψηφιακών Συστημάτων

Επίπεδο: Μεταπτυχιακό Πρόγραμμα Σπουδών

Ασφάλεια Δικτύων

Burp Suite

Επιβλέπον Καθηγητής: Καθ. Χρήστος Ξενάκης

Καλαϊτζίδης Ιωάννης

ioannis_kalaitzidis@ssl-unipi.gr

Πειραιάς
26/12/2025

Περίληψη

Η παρούσα εργασία εξετάζει την επιθετική προσέγγιση (Red Teaming) στην ασφάλεια διαδικτυακών εφαρμογών. Μέσω της πλατφόρμας PortSwigger-Web Security Academy και του εργαλείου Burp Suite, πραγματοποιήθηκε ανάλυση και εκμετάλλευση ευπαθειών σε ελεγχόμενο περιβάλλον. Συγκεκριμένα, υλοποιήθηκαν επιτυχώς επιθέσεις SQL injection για την ανάκτηση κρυφών δεδομένων και επιθέσεις cross-site scripting για την εκτέλεση αυθαίρετου κώδικα JavaScript, καταδεικνύοντας τη σημασία του ελέγχου εισόδου δεδομένων.

Abstract

This study examines the offensive approach (Red Teaming) in web application security. Using the PortSwigger-Web Security Academy platform and Burp Suite tool, vulnerabilities were analyzed and exploited in a controlled environment. Specifically, SQL Injection attacks were successfully implemented to retrieve hidden data, along with cross-site scripting attacks for executing arbitrary JavaScript code, demonstrating the critical importance of input validation mechanisms.

Περιεχόμενα

Εισαγωγή.....	1
Πρακτικό κομμάτι εργασίας.....	2
Βιβλιογραφία.....	16

Εισαγωγή

Η παρούσα εθελοντική εργασία διαφοροποιείται σημαντικά από την προηγούμενη εργασία (Case 7), καθώς σηματοδοτεί τη μετάβαση από την αμυντική (Blue Team) στην επιθετική (Red Team) προσέγγιση της ασφάλειας πληροφοριακών συστημάτων. Ενώ στο προηγούμενο σενάριο ο πρωταρχικός στόχος ήταν η θωράκιση του εξυπηρετητή μέσω μηχανισμών Firewall και WAF για την απόκρουση και τον αποκλεισμό κακόβουλης κίνησης, στην τρέχουσα μελέτη καλούμαστε να υιοθετήσουμε τη νοοτροπία και τις τεχνικές του επιτιθέμενου. Συγκεκριμένα, ο στόχος μετατοπίζεται από την προστασία στην ενεργή παραβίαση της εφαρμογής, επιδιώκοντας τον εντοπισμό και την εκμετάλλευση ευπαθειών. Για την πρακτική υλοποίηση των σεναρίων και την τεκμηρίωση των επιθέσεων SQL Injection και cross-site scripting μέσω του εργαλείου Burp Suite, θα χρησιμοποιηθεί το εξειδικευμένο εκπαιδευτικό περιβάλλον της PortSwigger-Web Security Academy. Η επιλογή αυτή κρίθηκε βέλτιστη καθώς παρέχει άμεση πρόσβαση σε ρεαλιστικά σενάρια ευπαθών εφαρμογών, επιτρέποντας την εστίαση στην ανάλυση των επιθέσεων χωρίς την πολυπλοκότητα και τους περιορισμούς που θα επέβαλε η εγκατάσταση και παραμετροποίηση ενός τοπικού ευάλωτου εξυπηρετητή. Τέλος, επισημαίνεται ότι τα επιλεγμένα εργαστήρια ανήκουν στην κατηγορία «Apprentice», καλύπτοντας πλήρως την απαίτηση της εκφώνησης για την τεκμηρίωση ευπαθειών χαμηλής (Low) δυσκολίας.

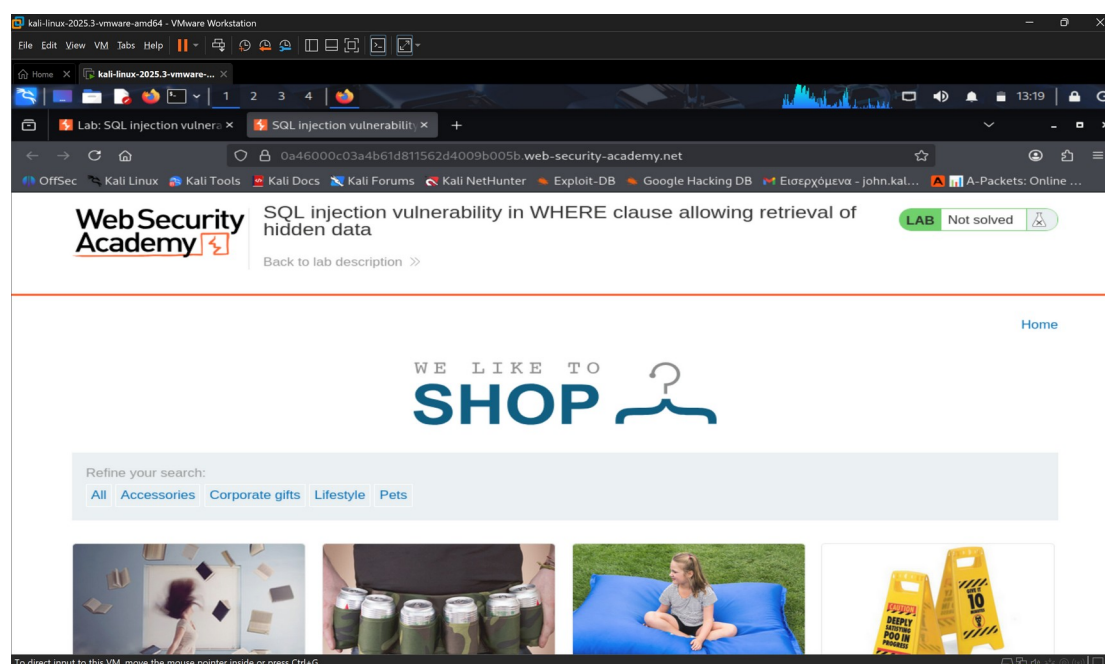
Πρακτικό κομμάτι εργασίας

SQL injection

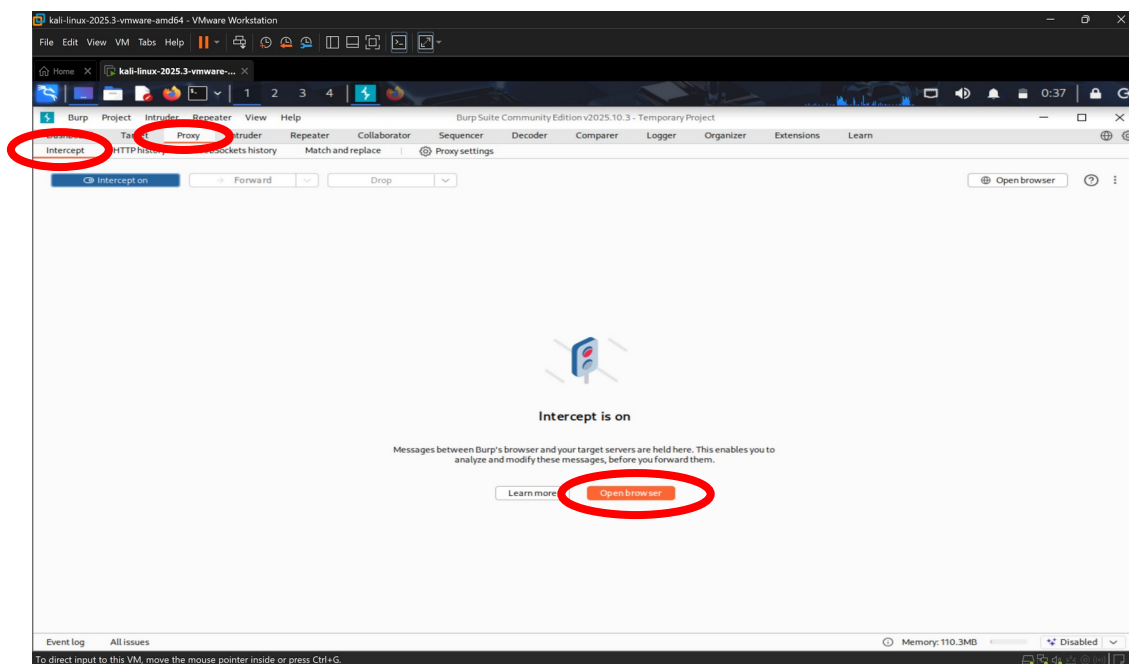
Για την υλοποίηση της εργασίας, επιλέχθηκε η εκπαιδευτική πλατφόρμα PortSwigger Web Security Academy, η οποία παρέχει ένα ασφαλές και ελεγχόμενο περιβάλλον για την εκτέλεση διαδικτυακών επιθέσεων. Συγκεκριμένα, πλοηγηθήκαμε στη θεματική ενότητα **SQL Injection** και επιλέξαμε το εργαστήριο με τίτλο:

«**SQL injection vulnerability in WHERE clause allowing retrieval of hidden data**» (<https://portswigger.net/web-security/sql-injection/lab-retrieve-hidden-data>). Η

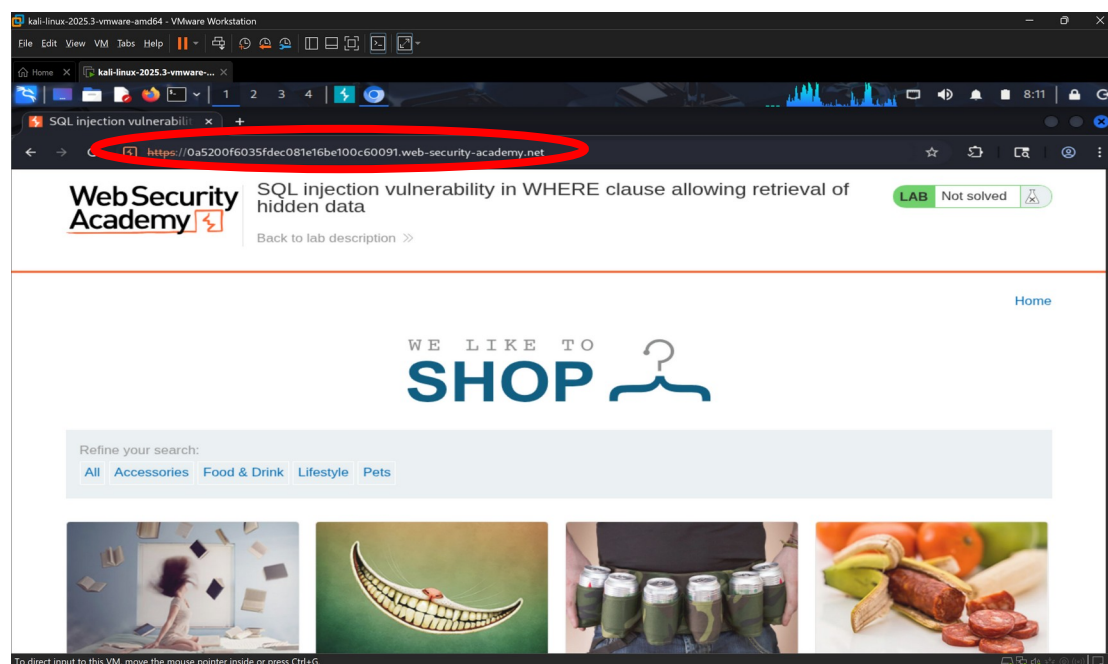
επιλογή του συγκεκριμένου εργαστηρίου κρίθηκε ως η βέλτιστη για το πρώτο σκέλος της άσκησης, καθώς προσομοιώνει ακριβώς το σενάριο που εξετάσαμε αμυντικά στην προηγούμενη εργασία (Case 7). Ενώ στο προηγούμενο σενάριο ο στόχος μας ήταν η ανίχνευση και η παρεμπόδιση της εντολής «OR 1=1» μέσω WAF, στο παρόν σενάριο ενεργούμε ως επιτιθέμενοι (Red Team). Στόχος μας είναι η επιτυχής εκτέλεση της ίδιας λογικής (1=1), προκειμένου να χειραγωγήσουμε το ερώτημα (query) προς τη βάση δεδομένων. Με τον τρόπο αυτό, επιδιώκουμε να παρακάμψουμε τους ελέγχους της εφαρμογής και να εξαναγκάσουμε την εμφάνιση κρυμμένων δεδομένων που υπό κανονικές συνθήκες δεν θα ήταν προσβάσιμα στον τελικό χρήστη.



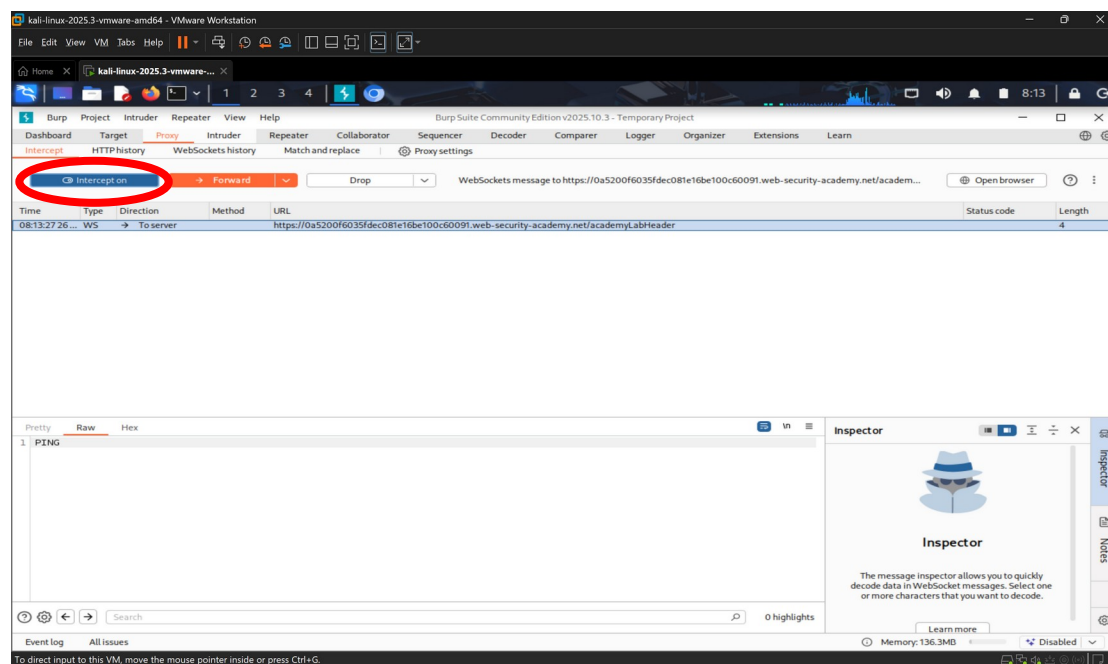
Οπότε ανοίγουμε το kali linux και αναζητούμε την εφαρμογή Burp Suite για να την ανοίξουμε.



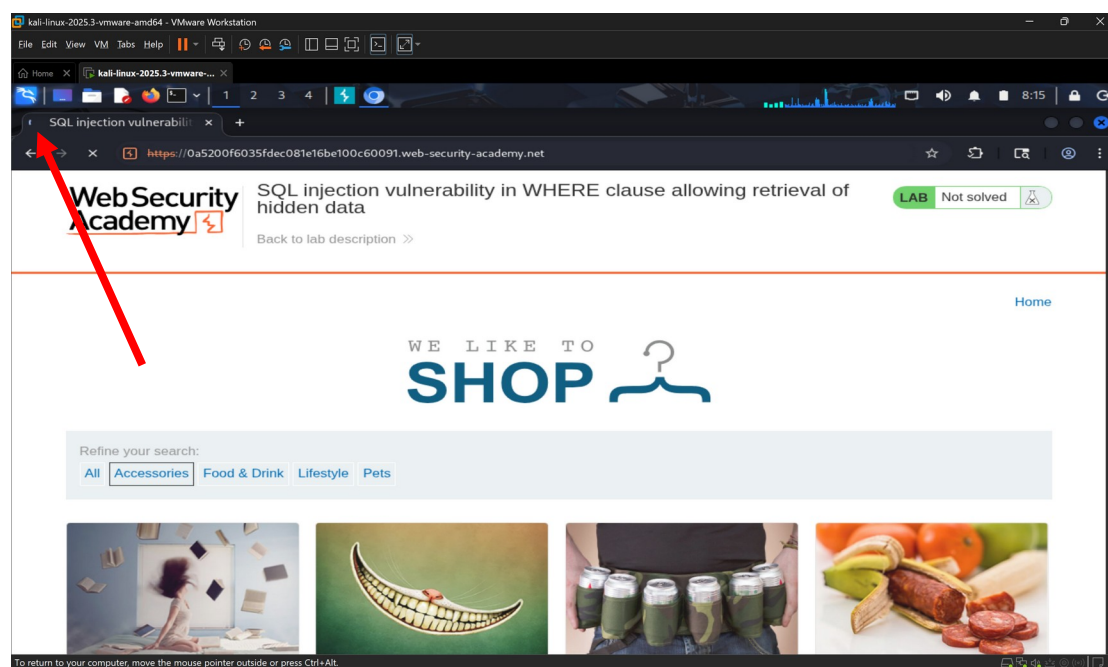
Αφού ανοίξει η εφαρμογή θα επιλέξουμε την καρτέλα **Proxy** και μετά την καρτέλα **Intercept**. Θα επιλέξουμε το κουμπί **Open Browser** που θα ανοίξει τον ειδικό browser της εφαρμογής και θα κάνουμε επικόλληση τη διεύθυνση από το συγκεκριμένο lab. Χρησιμοποιήσαμε τον ενσωματωμένο browser του Burp επειδή είναι έτοιμος για χρήση. Έτσι, γλιτώσαμε τις πολύπλοκες ρυθμίσεις δικτύου, καθώς η κίνηση περνάει αυτόματα μέσα από το εργαλείο.



Τώρα θα επιστρέψουμε στην εφαρμογή Burp Suite και θα κάνουμε ενεργοποίηση τη λειτουργία **Intercept on** όπως φαίνεται παρακάτω.

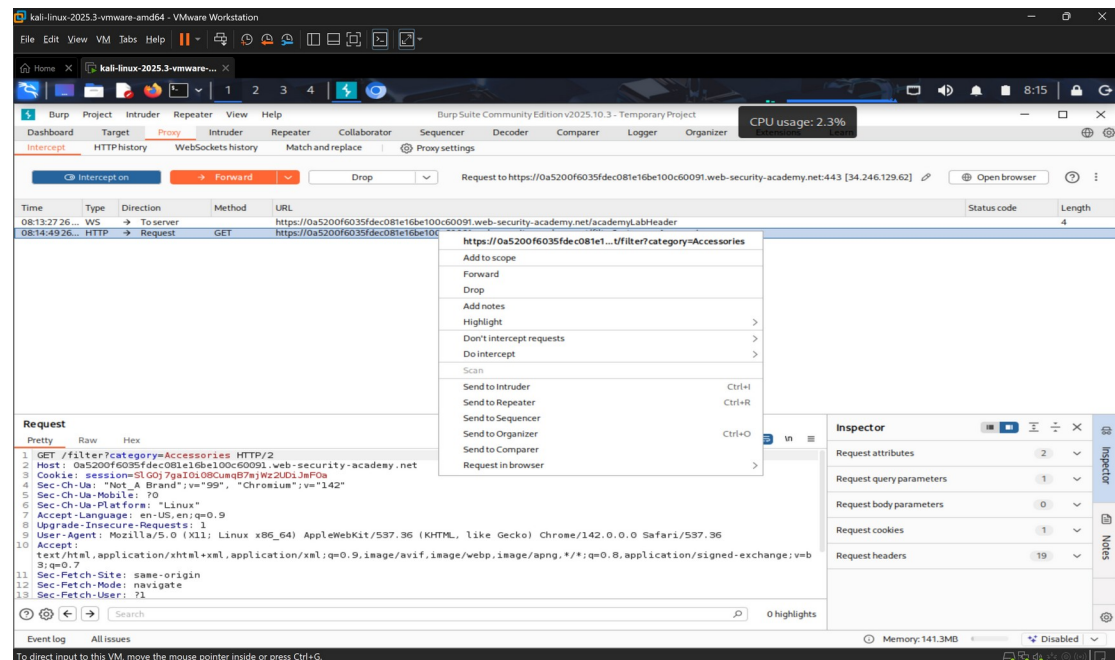


Αφού το κάνουμε αυτό, θα επιστρέψουμε στον browser του Burp:

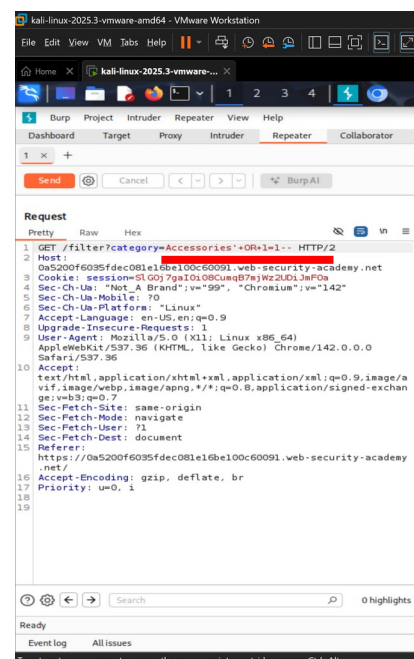
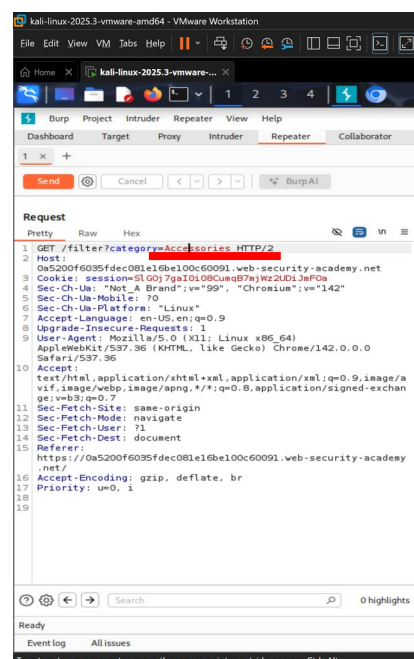


Στο στάδιο της αναγνώρισης, πλοηγούμαστε στην αρχική σελίδα του lab και αλληλοεπιδράμε με την εφαρμογή επιλέγοντας μία κατηγορία προϊόντων (συγκεκριμένα την κατηγορία «Accessories»). Η ενέργεια αυτή προκαλεί την αποστολή ενός HTTP αιτήματος προς τον εξυπηρετητή. Στόχος μας είναι να εντοπίσουμε και να αναχαιτίσουμε το συγκεκριμένο αίτημα μέσω του Burp Suite, ώστε

να ελέγξουμε αν η παράμετρος αυτή είναι ευάλωτη σε SQL Injection.



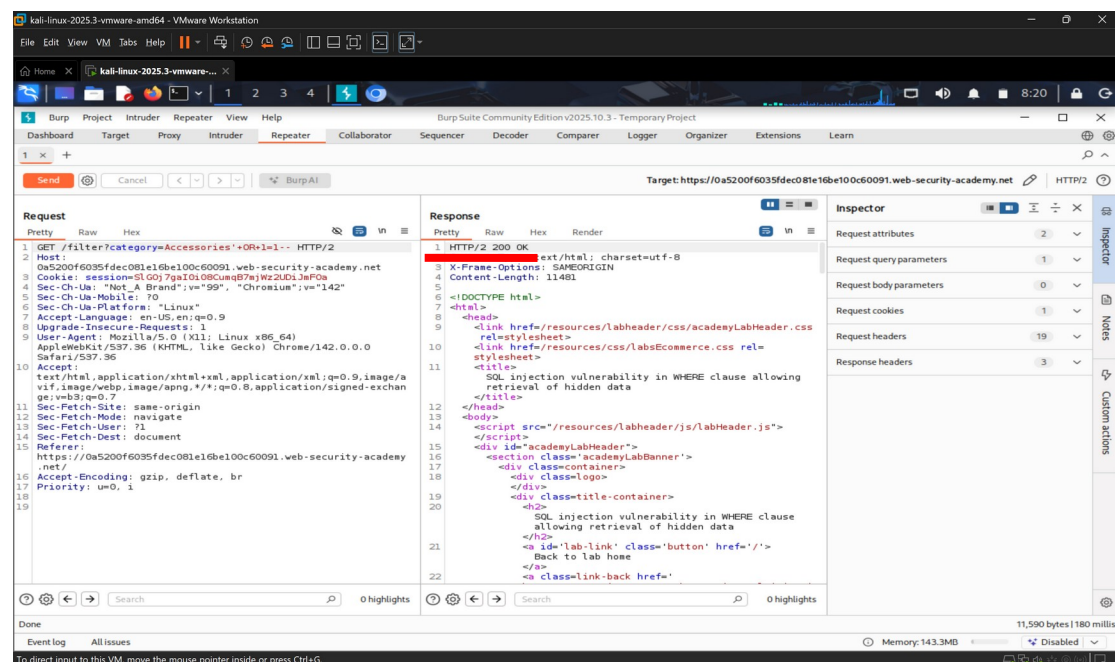
Αφού ενεργοποιήσουμε τη λειτουργία Intercept, το αίτημα HTTP αναχαιτίζεται επιτυχώς πριν φτάσει στον διακομιστή. Επιβεβαιώνουμε ότι πρόκειται για το αίτημα που καλεί τη σελίδα με τα αποτελέσματα της κατηγορίας «Accessories». Στη συνέχεια, κάνουμε δεξί κλικ στο αίτημα και επιλέξαμε «**Send to Repeater**». Με αυτό τον τρόπο μεταφέρει το αίτημα σε ένα περιβάλλον ελεγχόμενων δοκιμών, όπου μπορούμε να τροποποιήσουμε τις παραμέτρους και να επαναλάβουμε την αποστολή του αιτήματος πολλές φορές για να ελέγξουμε την αντίδραση του συστήματος στην επίθεση SQL Injection.



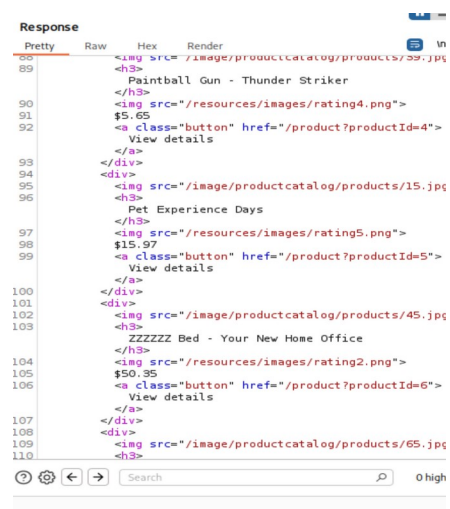
Εντοπίζουμε την παράμετρο category στη γραμμή του αιτήματος. Τροποποιούμε την τιμή της, αντικαθιστώντας το Accessories με το payload επίθεσης: «'OR 1=1 --».

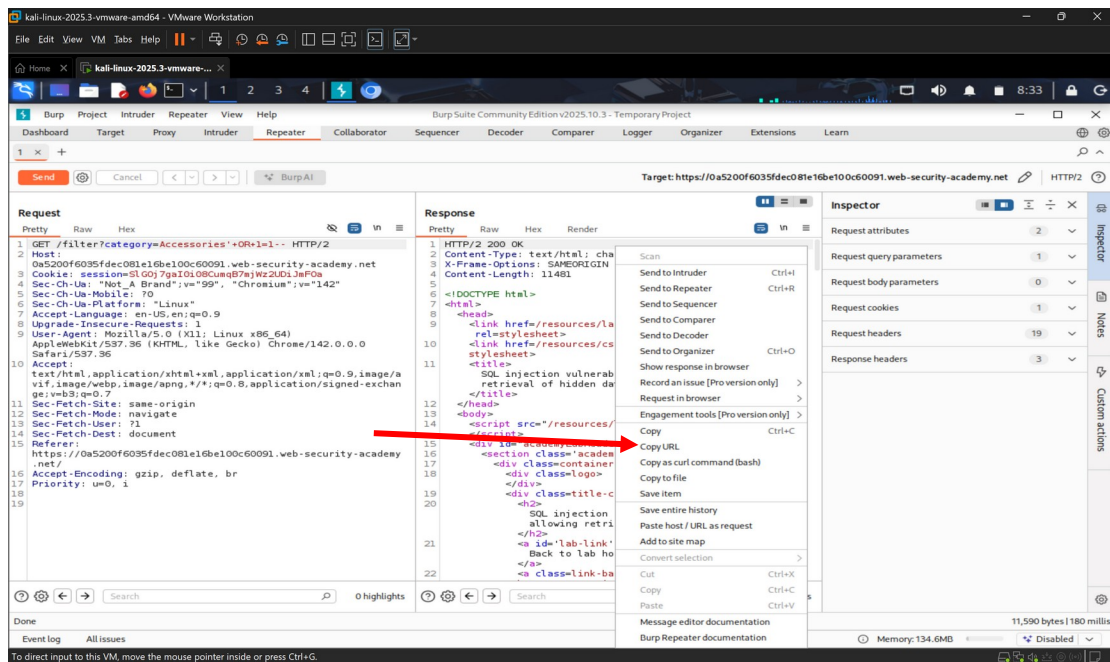
Ανάλυση Payload:

- Το μονό εισαγωγικό (') χρησιμοποιείται για να κλείσει το αλφαριθμητικό που περιμένει η βάση δεδομένων.
- Η συνθήκη OR 1=1 είναι μια λογική πρόταση που είναι πάντα αληθής.
- Οι παύλες (--) δηλώνουν σχόλιο στην SQL, ακυρώνοντας οποιοδήποτε μέρος του ερωτήματος ακολουθεί, ώστε να αποφευχθούν συντακτικά σφάλματα.



Όπως φαίνεται στο υπογραμμισμένο τμήμα της εικόνας (Response), ο διακομιστής απάντησε με κωδικό επιτυχίας **200 OK**. Αυτό υποδεικνύει ότι η SQL Injection επίθεση εκτελέστηκε σωστά και δεν προκάλεσε συντακτικό σφάλμα στη βάση δεδομένων. Παρατηρούμε επίσης ότι το μέγεθος της απάντησης αυξήθηκε, καθώς η ιστοσελίδα πλέον επιστρέφει δεδομένα για όλα τα προϊόντα της βάσης και όχι μόνο για τη συγκεκριμένη κατηγορία.

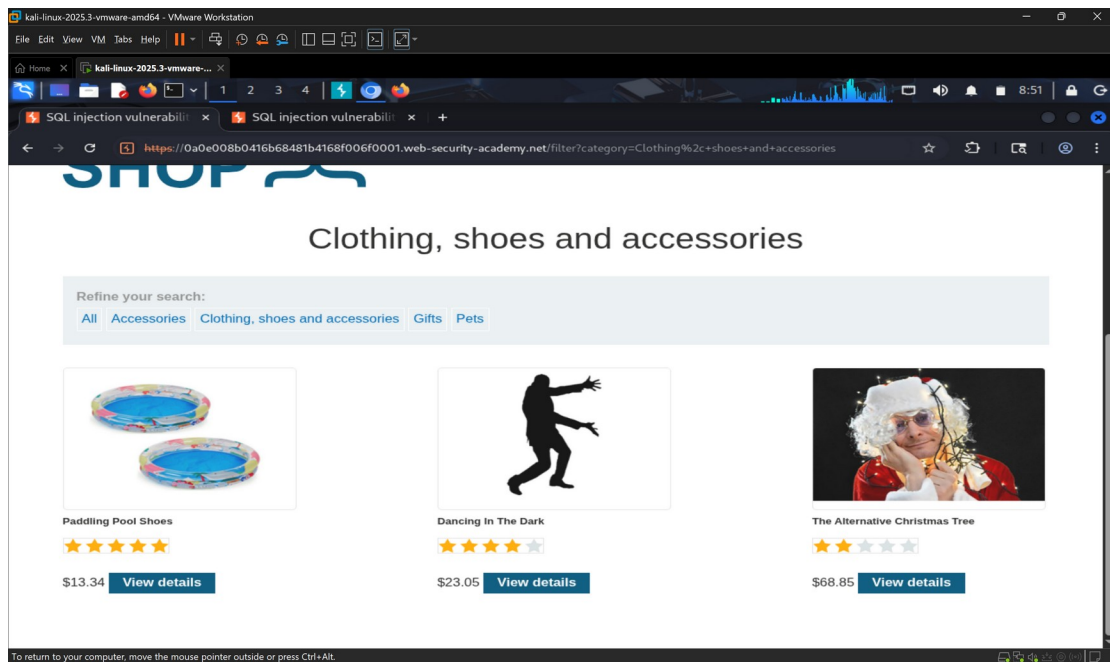




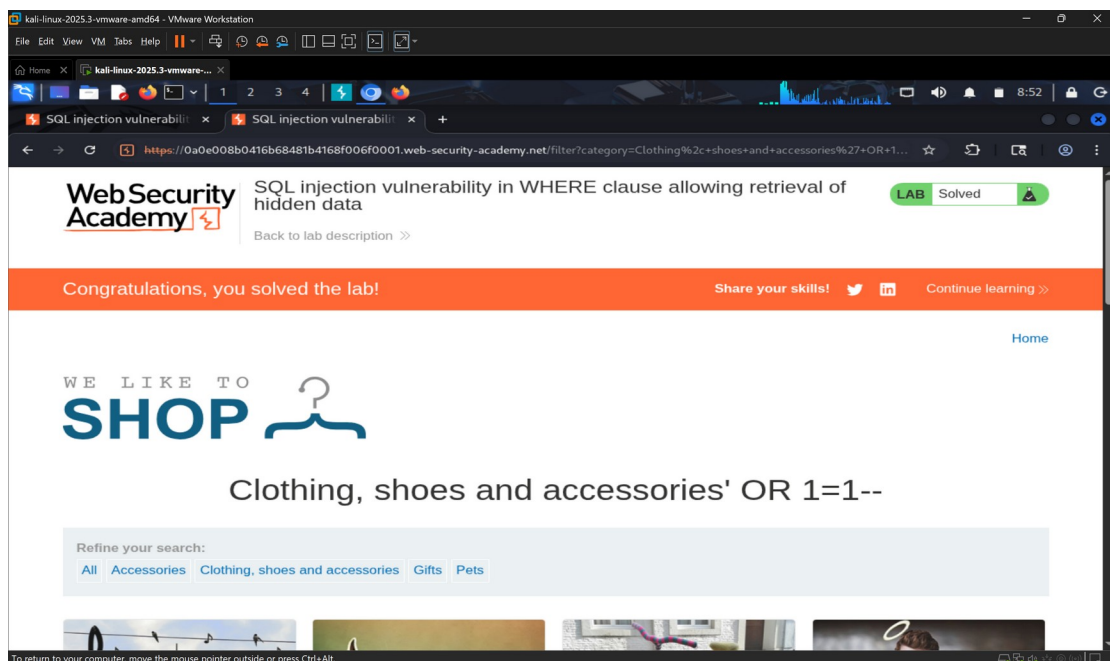
Έχοντας επιβεβαιώσει την επιτυχία της επίθεσης μέσω των κωδικών απάντησης στο repeater, το επόμενο βήμα ήταν η οπτική επαλήθευση στον browser. Για να το επιτύχουμε αυτό, απενεργοποιήσαμε τη λειτουργία **Intercept** στην καρτέλα Proxy του Burp Suite (που είχαμε ενεργοποιήσει νωρίτερα), ώστε να επιτρέψουμε την ελεύθερη ροή δεδομένων. Στη συνέχεια, χρησιμοποιήσαμε το τροποποιημένο URL (Copy URL) απευθείας στη γραμμή διευθύνσεων του browser.

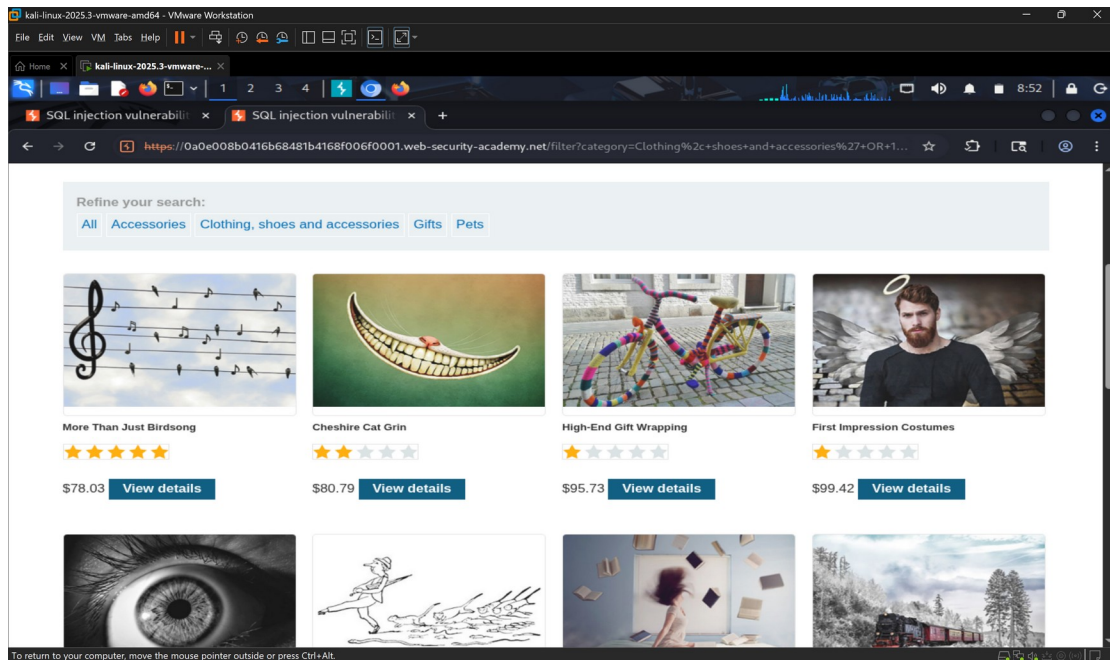
Για την τελική επαλήθευση, συγκρίναμε τη συμπεριφορά της εφαρμογής πριν και μετά την επίθεση. Στην αρχική αναζήτηση (εικόνα που ακολουθεί), η εφαρμογή φιλτράρει σωστά τα αποτελέσματα, εμφανίζοντας μόνο τα προϊόντα που ανήκουν στην

κατηγορία «Clothing, shoes and accessories».



Μετά την εισαγωγή του payload $OR\ 1=1$, παρατηρούμε στα αποτελέσματα εμφανίζονται πλέον όλα τα προϊόντα της βάσης δεδομένων, ανεξαρτήτως κατηγορίας, καθώς και προϊόντα που πιθανώς ήταν κρυμμένα/μη δημοσιευμένα.





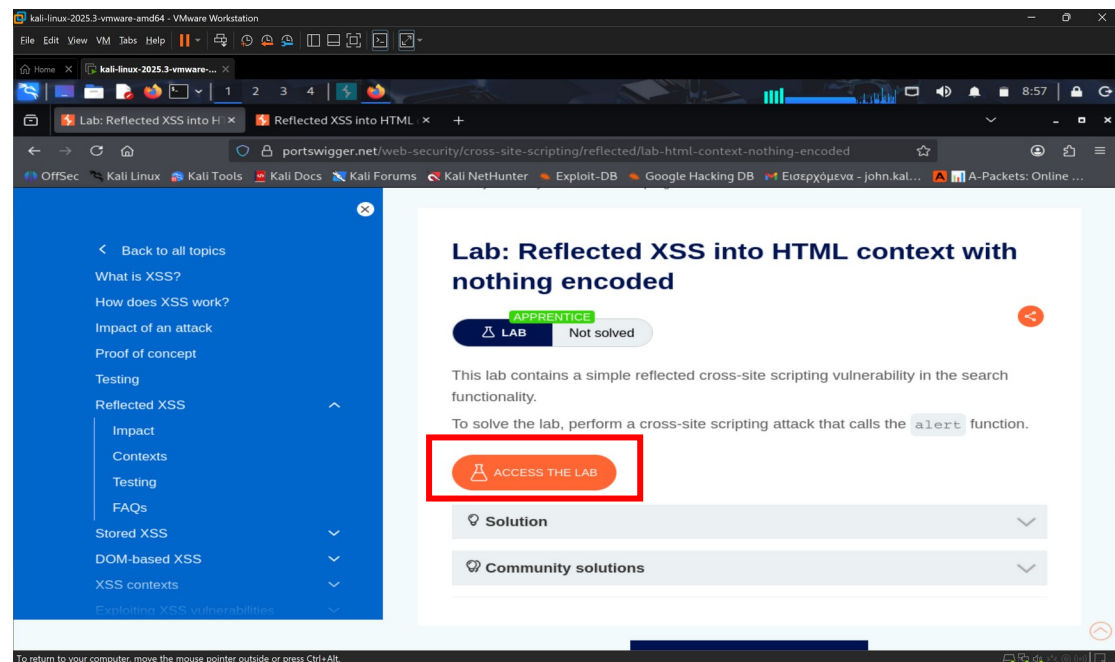
Συμπέρασμα Ενότητας SQL Injection

Μέσω της συγκεκριμένης επίθεσης, αποδείχθηκε ότι η εφαρμογή δεν πραγματοποιεί επαρκή έλεγχο στα δεδομένα που εισάγει ο χρήστης. Εισάγοντας τη συνθήκη 'OR 1=1--, καταφέραμε να τροποποιήσουμε τη λογική του SQL ερωτήματος (query) προς τη βάση δεδομένων. Επειδή η συνθήκη 1=1 αξιολογείται πάντα ως αληθής, η βάση δεδομένων σταμάτησε να ελέγχει την κατηγορία του προϊόντος και επέστρεψε όλες τις εγγραφές του πίνακα. Αυτό συνιστά σοβαρή παραβίαση της εμπιστευτικότητας, καθώς επιτρέπει σε μη εξουσιοδοτημένους χρήστες να αποκτήσουν πρόσβαση σε δεδομένα που ο διαχειριστής είχε ορίσει ως κρυφά ή περιορισμένα.

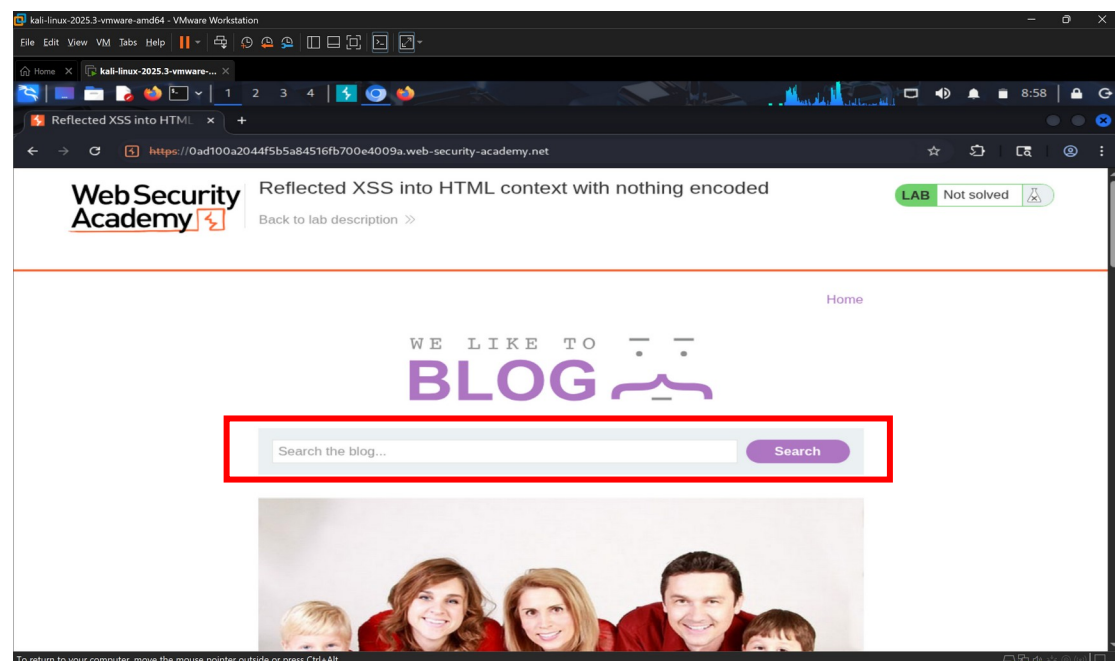
Cross-site scripting attacks

Για το δεύτερο μέρος της εργασίας, μεταβήκαμε στην κατηγορία επιθέσεων Cross-Site Scripting (XSS). Συγκεκριμένα, επιλέξαμε το εργαστήριο με τίτλο «Reflected XSS into HTML context with nothing encoded» (<https://portswigger.net/web-security/cross-site-scripting/reflected/lab-html-context-nothing-encoded>). Στόχος της συγκεκριμένης άσκησης είναι να εντοπίσουμε ένα σημείο εισόδου (input vector) όπου η εφαρμογή «αντικατοπτρίζει» τα δεδομένα του

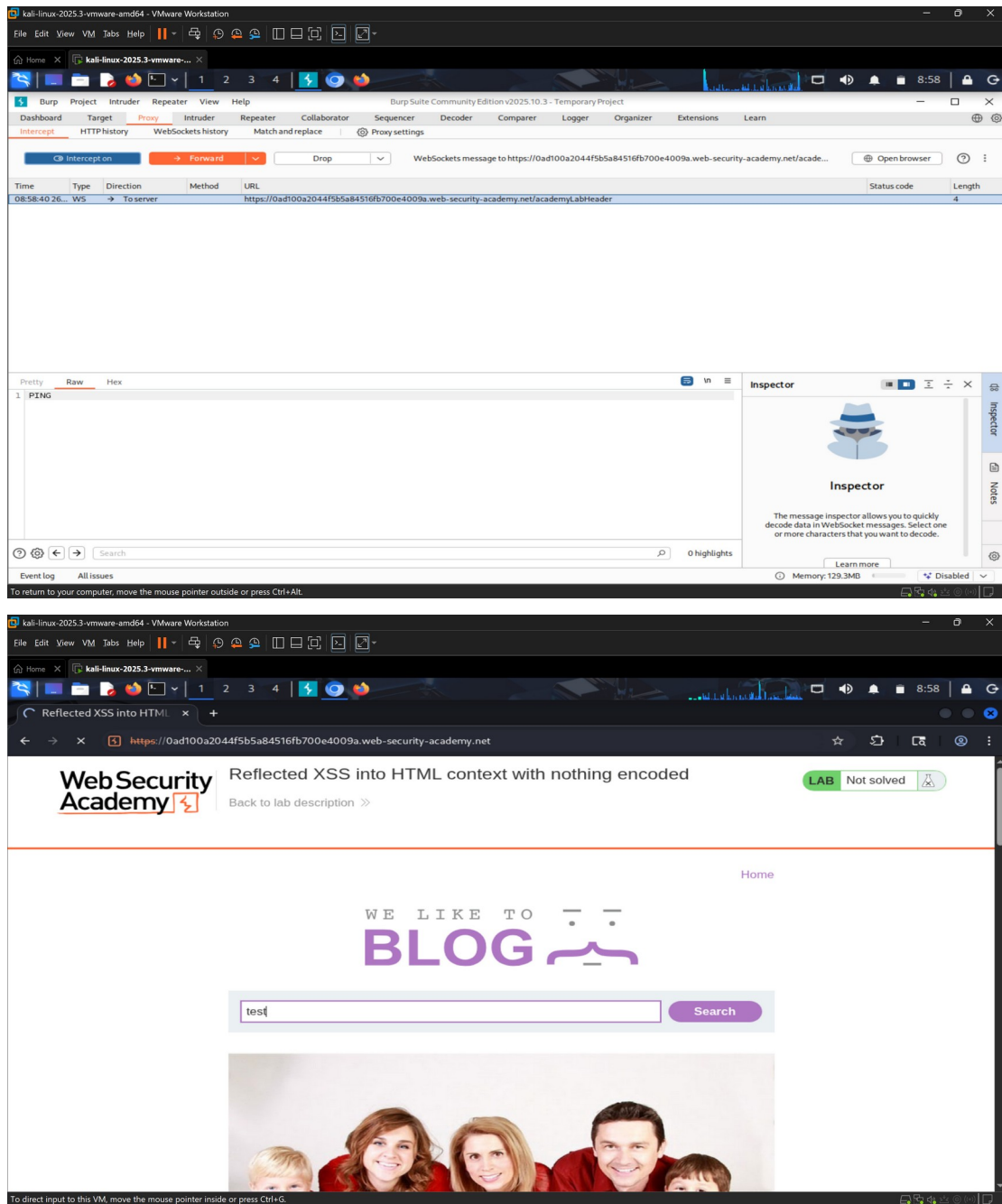
χρήστη πίσω στον browser χωρίς κατάλληλο φιλτράρισμα. Αυτό θα μας επιτρέψει να εισάγουμε και να εκτελέσουμε αυθαίρετο κώδικα JavaScript.



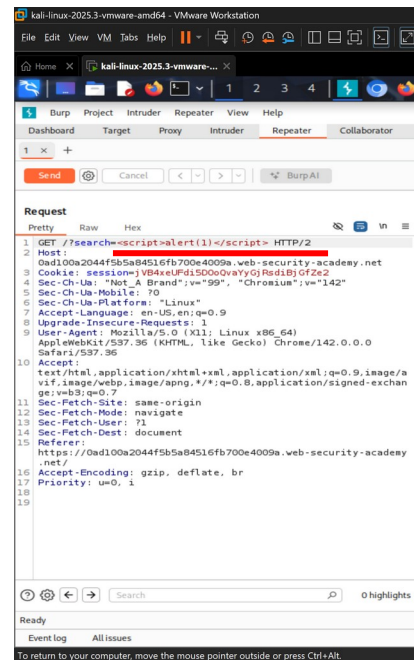
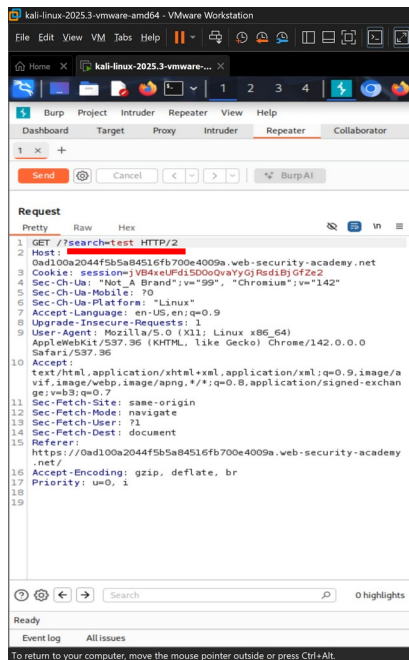
Πατώντας «Access the Lab», μεταφερθήκαμε στο περιβάλλον της ευάλωτης εφαρμογής, η οποία προσομοιώνει ένα blog. Κατά τη φάση της αναγνώρισης, εντοπίσαμε τη μπάρα αναζήτησης.



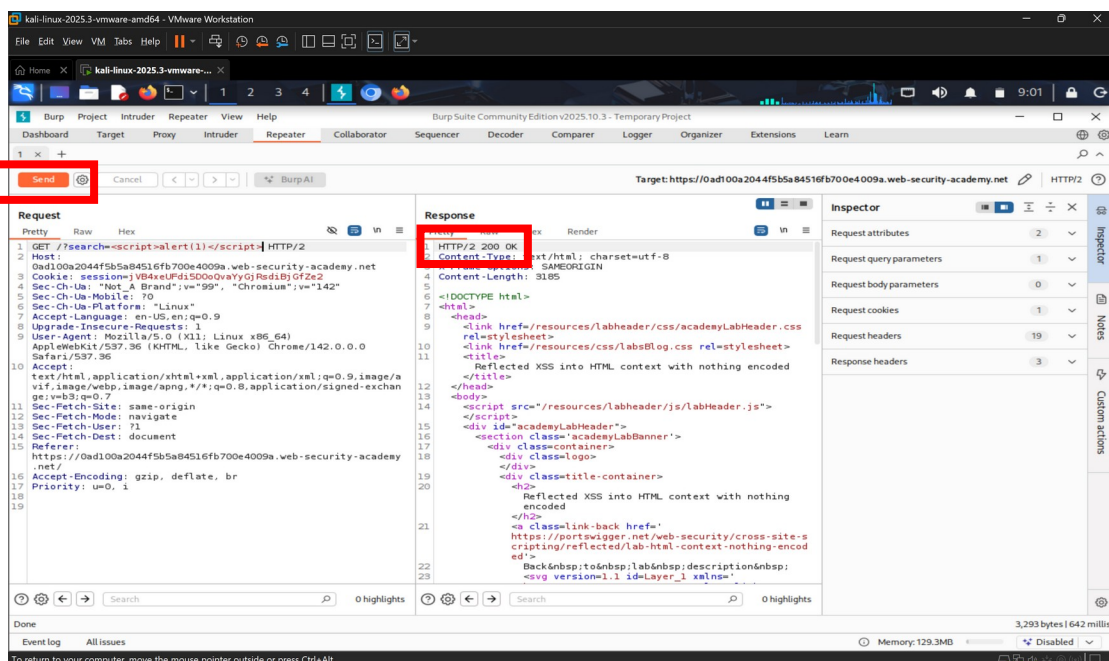
Η μπάρα αναζήτησης αποτελεί το πρωταρχικό σημείο ενδιαφέροντος για επίθεση reflected XSS, καθώς είναι σύνηθες τα δεδομένα που πληκτρολογεί ο χρήστης να εμφανίζονται αυτούσια στη σελίδα αποτελεσμάτων. Επόμενο βήμα είναι η δοκιμή του πεδίου μέσω του Burp Suite.



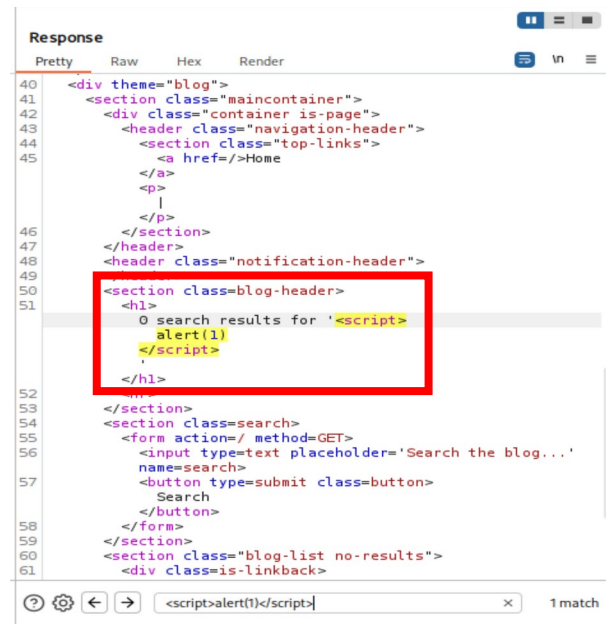
Για την ανάλυση της κίνησης, ακολουθήσαμε την ίδια μεθοδολογία με την προηγούμενη ενότητα (SQL Injection). Συγκεκριμένα, ενεργοποιήσαμε τη λειτουργία «**Intercept is On**» στο Burp Suite και πραγματοποιήσαμε μια αναζήτηση στην εφαρμογή για να δημιουργήσουμε κίνηση. Το αίτημα αναχαιτίστηκε επιτυχώς, επιτρέποντάς μας να το εξετάσουμε και να το τροποποιήσουμε πριν φτάσει στον εξυπηρετητή.



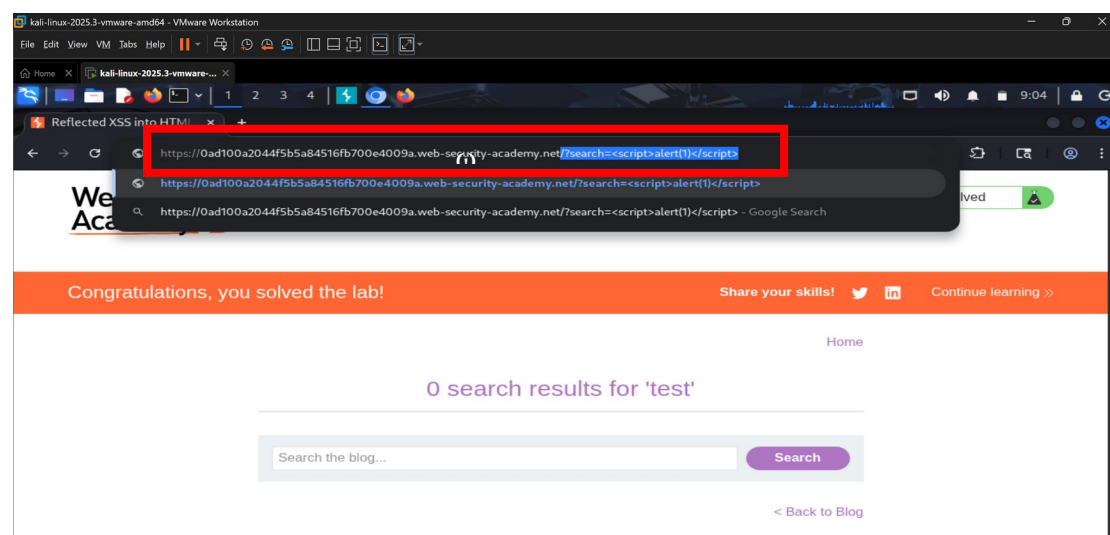
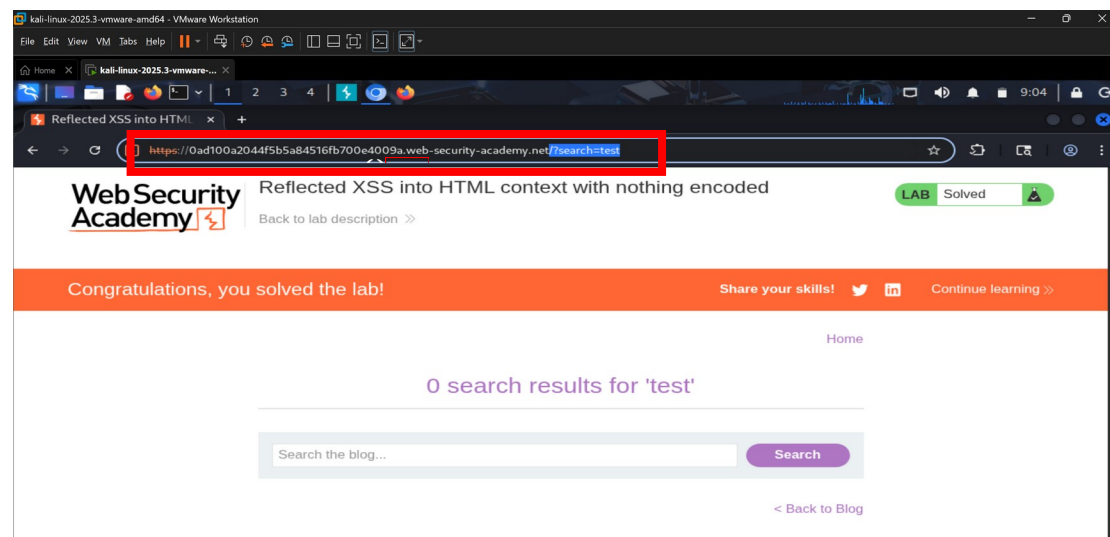
Αφού εντοπίσαμε το αίτημα που μεταφέρει τον όρο αναζήτησης, το στείλαμε στο εργαλείο Repeater για να πραγματοποιήσουμε δοκιμές χωρίς να επηρεάσουμε τον browser. Στο Repeater, αντικαταστήσαμε τον αρχικό όρο αναζήτησης (**test**) με το κλασικό payload δοκιμής για XSS «**<script>alert(1)</script>**». Σκοπός αυτού του script είναι να εμφανίσει ένα αναδυόμενο παράθυρο (alert box) με τον αριθμό «1», εφόσον ο browser εκτελέσει τον κώδικα.



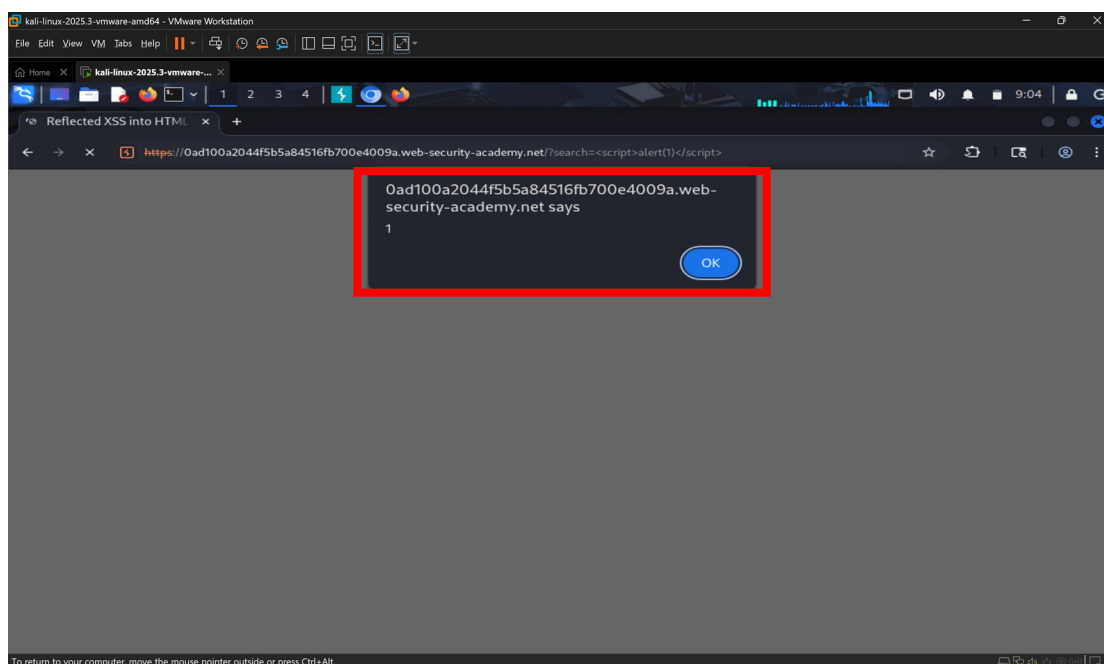
Πατώντας **send**, λάβαμε την απάντηση του διακομιστή. Εξετάζοντας τον κώδικα HTML στο δεξί μέρος (Response), παρατηρήσαμε ότι η εφαρμογή επέστρεψε το script μας ακριβώς



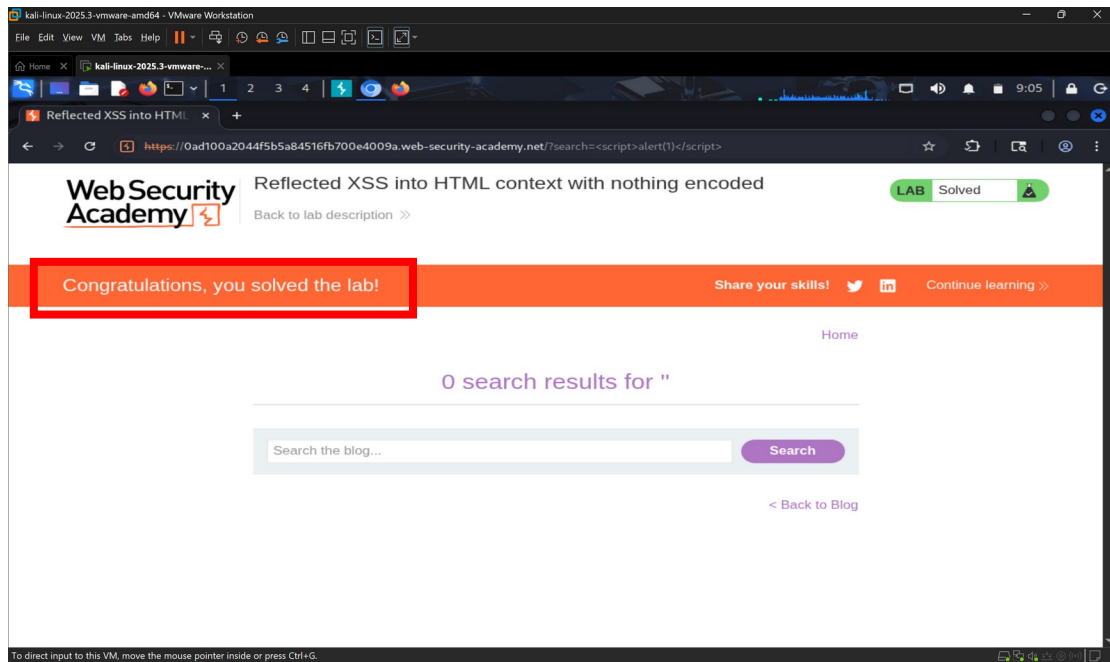
```
Response
Pretty Raw Hex Render
40 <div theme="blog">
41 <section class="maincontainer">
42 <div class="container is-page">
43 <header class="navigation-header">
44 <section class="top-links">
45 <a href=/>Home
46 </a>
47 <p>
48 </p>
49 </section>
50 </header>
51 <header class="notification-header">
52 <section class="blog-header">
53 <h1>
54 0 search results for '<script>
55 alert(1)
56 </script>'
57 </h1>
58 </section>
59 </section>
60 <section class="search">
61 <form action=/ method=GET>
62 <input type=text placeholder='Search the blog...'
63 name=search>
64 <button type=submit class=button>
65 Search
66 </button>
67 </form>
68 </section>
69 <section class="blog-list no-results">
70 <div class=is-linkback>
```



Έχοντας επιβεβαιώσει την ευπάθεια στο repeater, προχωρήσαμε στην τελική εκτέλεση της επίθεσης σε πραγματικό περιβάλλον περιηγητή. Πρώτα, μεταβήκαμε στην καρτέλα Proxy και απενεργοποιήσαμε τη λειτουργία **Intercept is Off**, ώστε να επιτραπεί η ελεύθερη ροή δεδομένων. Στη συνέχεια, ανοίξαμε τον browser και στη γραμμή διευθύνσεων τροποποιήσαμε χειροκίνητα την παράμετρο αναζήτησης, εισάγοντας το payload «**/?search=<script>alert(1)</script>**»



Πατώντας enter στη γραμμή διευθύνσεων, ο browser έστειλε το αίτημα και έλαβε την απάντηση που περιείχε το script μας. Όπως φαίνεται στην εικόνα, ο browser ερμήνευσε τα tags `<script>` ως εντολή προς εκτέλεση και όχι ως απλό κείμενο. Το αποτέλεσμα ήταν η εμφάνιση ενός αναδυόμενου παραθύρου διαλόγου (pop-up alert) με την τιμή «1». Η εμφάνιση αυτού του παραθύρου αποτελεί την αδιαμφισβήτητη απόδειξη ότι η εφαρμογή είναι ευάλωτη σε Cross-Site Scripting, καθώς επιτρέπει την εκτέλεση JavaScript στον υπολογιστή του χρήστη.



Μετά το κλείσιμο του παραθύρου διαλόγου, η σελίδα φόρτωσε πλήρως και η πλατφόρμα εμφάνισε το μήνυμα «Congratulations, you solved the lab!». Αυτό επιβεβαιώνει και από την πλευρά της Web Security Academy ότι η επίθεση πραγματοποιήθηκε σωστά και ο στόχος επιτεύχθηκε.

Συμπέρασμα ενότητας cross-site scripting attacks

Στο σενάριο αυτό, διαπιστώσαμε ότι η εφαρμογή λαμβάνει δεδομένα από τον χρήστη με την μπάρα αναζήτησης και τα επιστρέφει άμεσα στον κώδικα της σελίδας (HTML). Αυτό το κενό ασφαλείας επιτρέπει σε κακόβουλους χρήστες να εισάγουν αυθαίρετο κώδικα JavaScript. Αν και στο παράδειγμά μας χρησιμοποιήσαμε μια αβλαβή εντολή `alert()`, σε πραγματικές συνθήκες ένας επιτιθέμενος θα μπορούσε να χρησιμοποιήσει την ίδια ευπάθεια για να υποκλέψει τα cookies, οδηγώντας σε παραβίαση λογαριασμών χρηστών.

Βιβλιογραφία

- PortSwigger. (n.d.). Burp Suite Community Edition. <https://portswigger.net/burp/communitydownload>
- PortSwigger Web Security Academy. (n.d.). Lab: SQL injection vulnerability in WHERE clause allowing retrieval of hidden data. <https://portswigger.net/web-security/sql-injection/lab-retrieve-hidden-data>
- PortSwigger Web Security Academy. (n.d.). Lab: Reflected XSS into HTML context with nothing encoded. <https://portswigger.net/web-security/cross-site-scripting/reflected/lab-html-context-nothing-encoded>
- PortSwigger. (n.d.). SQL injection. <https://portswigger.net/web-security/sql-injection>
- PortSwigger. (n.d.). Cross-site scripting (XSS). <https://portswigger.net/web-security/cross-site-scripting>
- Seven Seas Security. (2022, May 5). Cross-Site Scripting Lab Breakdown: Reflected XSS into HTML context with nothing encoded. <https://www.youtube.com/watch?v=HfBiMStbWqY>
- David Bombal Clips. (2024, June 11). Mastering Burp Suite: the ultimate web application hacking tool. <https://www.youtube.com/watch?v=r46b9L6UQDo>
- Netsec Explained. (2023, September 20). Master Burp Suite like a pro in just 1 hour. <https://www.youtube.com/watch?v=QiNLNDSLujY>